

Laboratório de Concorrência · Aula 10

Enzo Seiti Hashimoto · RA 24190357-2 · Turma esoft6c

Gerado em 09/10/2026, 20:42:21 · tempo registrado no cronômetro: 78:05

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação: Uma tarefa A deve esperar a tarefa B terminar de fazer sua ação, ex: devo esperar uma pessoa ser atendida para eu ser atendido.

Competição: Duas tarefas que precisam usar mesmo recurso não podem ser usados ao mesmo tempo, ex: Dois atendimentos de clientes ao mesmo tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000 acredito

1ª execução (n = 1.000.000)

1331305

2ª execução (n = 1.000.000)

1346420

3ª execução (n = 1.000.000)

1323643

Saída do programa

```
esperado: 2000000
obtido: 1331305
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram. Deram 3 resultados diferentes. Esse fenômeno ocorre por culpa das condições de corrida, em que as duas threads acessam a mesma variável sem sincronização, assim fazendo uns números se perderem.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Uma thread (A) lê uma variável e seu valor, ex: $i = 10$; A outra thread (B) lê essa mesma variável i , em seguida a (A) vai somar 1 para a variável i ($10 + 1$), porém a thread (B) não lê o valor atualizado e adiciona 1 a $i = 10$; ($10 + 1$). Por fim ambas threads gravam o valor 11 da variável i .

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Usando $n = 1000$ o valor esperado bateu com o obtido (2000), porém mesmo gerando resultado correto existem ainda assim os riscos de gerar uma condição corrida, que podem ser evitados se usarmos mecanismos de

sincronização para fazer updates de forma segura.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

não respondido

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens de execução e aparecem 4 resultados diferentes: 4, 6, 7 e 8. Apenas 2 ordens produzem o resultado correto, que é 8: AAABBB e BBBAAA.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBB, a tarefa A lê o valor 3 e soma 1, obtendo 4, mas ainda não grava o resultado. Em seguida, a tarefa B lê o valor 3, multiplica por 2 e grava 6 em TOTAL. Por último, A grava o resultado 4, sobrescrevendo o valor 6 que B havia gravado. Assim, o resultado final é 4, porque a atualização feita por B foi perdida. Isso acontece devido à condição de corrida entre as duas tarefas.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Operações atômicas, já que, ele faz todas operações juntas em um passo só, não acho que alguma será incorreta, mas acredito que operações atômicas sacrificam segurança em troca de mais velocidade

Saída do programa

```
long ms2 = mede(() → { for (int i = 0; i < n; i++) incSync(); });
long ms3 = mede(() → { for (int i = 0; i < n; i++) atomic.incrementAndGet(); });
System.out.println("esperado:          " + 2 * n);
System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
System.out.println("synchronized:      " + comSync + " em " + ms2 + " ms");
System.out.println("AtomicInteger:      " + atomic.get() + " em " + ms3 + " ms");
}
}
1.1s
esperado:          2000000
sem sincronizacao: 1021959 em 2 ms
synchronized:      2000000 em 42 ms
AtomicInteger:      2000000 em 44 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Synchronized e AtomicInterger. A versão correta garante que o código rode com mais segurança, controlando o acesso à variável compartilhada ou garantindo que cada incremento seja atômico, assim gerando um custo adicional a mais de execução.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Usar um AtomicInteger para cada saldo garante que cada saldo seja atualizado individualmente de forma atômica, mas não garante que a retirada de uma conta e o depósito na outra aconteçam como uma única operação. Para evitar inconsistências, seria necessário sincronizar a transferência inteira, garantindo que as duas operações sejam concluídas juntas, ou que nenhuma delas seja aplicada caso ocorra uma falha.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Sim

Saídas (antes e depois da correção)

Antes: IMPASSE: 2 threads presas

Depois: Novo código da Thread b:

```
Thread b = new Thread(() → {  
    synchronized (X) {  
        pausa();  
        synchronized (Y) {  
            System.out.println("B terminou");  
        }  
    }  
});
```

Resultado:

A terminou

B terminou

sem impasse

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque a thread A pegou o recurso X e ficou esperando o recurso Y, enquanto a thread B pegou o recurso Y e ficou esperando o recurso X. Como cada uma estava segurando um recurso que a outra precisava, nenhuma conseguia acessar seus recursos que precisavam, causando um impasse / deadlock.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem elimina esse tipo de impasse porque impede que uma thread fique esperando um recurso enquanto segura outro adquirido em uma ordem contrária. Assim, se as duas threads precisarem de X e Y, ambas tentarão pegar primeiro X e depois Y. Um exemplo real seria um sistema bancário em que duas transferências simultâneas tentam acessar duas contas em ordens opostas, podendo ficar bloqueadas se não houver uma ordem padronizada de acesso.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

6 tarefas de 100 = 600, entre 3 trabalhadores é esperado 200ms de cada, se 1 trabalhador fizer todo o trabalho então é esperado 600ms dele, dividido entre os 6 trabalhadores é esperado 100ms de cada

Tempo com 1 trabalhador

1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

Tempo com 3 trabalhadores

3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

Tempo com 6 trabalhadores

6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

Saída do programa

```
1:
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
3:
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 4
trabalhador 2 fez a tarefa 5
trabalhador 3 fez a tarefa 6
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
6:
trabalhador 1 fez a tarefa 1
trabalhador 5 fez a tarefa 5
trabalhador 6 fez a tarefa 6
trabalhador 3 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 4 fez a tarefa 4
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Os tempos variam conforme o número de trabalhadores: quanto mais trabalhadores executam tarefas em paralelo, menor tende a ser o tempo total. Com 1 trabalhador, as tarefas são realizadas sequencialmente; com 3, são divididas entre três trabalhadores; e com 6, todas podem ser executadas simultaneamente.

O programa não precisa de synchronized nem de travas porque as tarefas são distribuídas por meio de um canal (channel), que permite a comunicação entre as goroutines. Cada tarefa é recebida por apenas um trabalhador, evitando que várias goroutines precisem modificar a mesma variável compartilhada. O WaitGroup apenas espera que todos terminem.

Saídas (antes e depois da troca)

Antes:

```
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]:
```

```
main.main()
```

```
    /box/main.go:9 +0x38
```

```
exit status 2
```

Depois:

```
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não tem espaço para armazenar mensagens. Ao executar `ch <- 1`, o programa fica esperando que outra goroutine receba o valor, mas não existe nenhuma outra goroutine para fazer isso. Por isso, ocorre um erro de deadlock.

Quando trocamos para `make(chan int, 1)`, o canal passa a ter espaço para armazenar uma mensagem. Assim, o valor 1 é armazenado no canal sem precisar que alguém o receba imediatamente. Em seguida, a instrução `fmt.Println(<-ch)` recebe o valor e o imprime normalmente.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Erro na compilação provavelmente

Saídas (o erro e o resultado com move)

Antes:

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned by the current function
  → HelloWorld.rs:7:27
```

Depois:

```
1
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com `move`, a thread passa a ser dona do valor `contador`, levando o número para dentro da thread. Assim, ela altera sua própria cópia, enquanto a variável original de `main` continua valendo 0. Por isso, o programa imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Na Prática 1, em Java, o erro de condição de corrida aparece durante a execução e pode passar despercebido nos testes, sendo descoberto somente depois, em produção. Em Rust, o compilador identifica o problema de compartilhamento inseguro antes de o programa executar.

A vantagem do Rust é prevenir determinados erros de concorrência antes da execução, tornando o programa mais seguro. O custo é que o programador precisa aprender e respeitar regras de propriedade, empréstimo e sincronização, o que pode tornar o desenvolvimento inicialmente mais complexo.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A
C
D
B

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 301 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem de impressão foi A, D, C e B. A letra A aparece primeiro porque é impressa diretamente. Depois, D também é impressa diretamente, antes de o código terminar. Em seguida, C aparece porque as promessas são executadas na fila de microtarefas, antes dos temporizadores. Por último, B aparece porque o setTimeout agenda uma tarefa para ser executada posteriormente. Mesmo com tempo 0, ela precisa esperar o código atual e as microtarefas terminarem.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms executou bem depois desse tempo porque a função ocupado(300) bloqueou a única thread por aproximadamente 300 ms, impedindo que o laço de eventos processasse outras tarefas nesse período. Isso significa que, em um servidor Node.js, um cálculo pesado executado diretamente pode bloquear o atendimento de outros usuários, atrasando respostas e temporizadores. Para evitar isso, é possível dividir o cálculo em tarefas menores ou utilizar Worker Threads para executar operações pesadas em paralelo.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo.

Qual problema desta atividade pode acontecer? Como você o evitaria?

O problema seria vender o mesmo ingresso para duas pessoas, causando uma inconsistência no sistema. Para evitar isso, seria necessário proteger a operação de verificar a disponibilidade e reservar o ingresso com uma transação atômica ou um mecanismo de sincronização, garantindo que apenas uma compra seja confirmada para o último lugar.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi rever novamente os conceitos da matéria de Sistema Operacionais e como ela impacta nessa nova matéria de Paradigmas de software.